CS 301
High-Performance Computing

Lab 01 - CPU architecture, Memory Hierarchy, and Performance Measurement

Prasham Shah (202301252)
Dhruvi Jobanputra (202301401)

January 28, 2026

Contents

1 Introduction

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

2.2 Hardware Details for HPC Cluster (Node gics1)

3 Problem Description

4 Graphical Results

4.1 Vector Copy Operation : $a[i] = b[i]$

4.1.1 Throughput and Bandwidth using Lab PC

4.1.2 Throughput and FLOPs using HPC Cluster

4.2 Vector Scaling Operation : $a[i] = k \times b[i]$

4.2.1 Throughput and Bandwidth using Lab PC

4.2.2 Throughput and FLOPs using HPC Cluster

4.3 Vector Sum Operation : $a[i] = b[i] + c[i]$

4.3.1 Throughput and Bandwidth using Lab PC

4.3.2 Throughput and FLOPs using HPC Cluster

4.4 Vector Triad Operation : $a[i] = b[i] + c[i] \times d[i]$

4.4.1 Throughput and Bandwidth using Lab PC

4.4.2 Throughput and FLOPs using HPC Cluster

5 Analytical Tasks

6 Conclusion

7 Introduction

In this experiment, we evaluate the CPU performance of both the laboratory machine and the computing cluster. To understand how system architecture affects performance, we implement the vector triad operation along with its related variants such as copy, scale, and add. These vector operations were introduced in the theory sessions, and through this lab we analyze their practical behavior and observe how they influence execution time and overall performance.

8 Hardware Details

8.1 Hardware Details for LAB207 PCs

- **Architecture:** x86_64
- **CPU op-mode(s):** 32-bit, 64-bit
- **Address sizes:** 46 bits physical, 48 bits virtual
- **Byte Order:** Little Endian
- **CPU(s):** 12
- **On-line CPU(s) list:** 0–11
- **Thread(s) per core:** 2
- **Core(s) per socket:** 6
- **Socket(s):** 1
- **NUMA node(s):** 1
- **NUMA node0 CPU(s):** 0–11
- **Vendor ID:** GenuineIntel
- **CPU family:** 6
- **Model:** 151
- **Model name:** 12th Gen Intel(R) Core(TM) i5-12500
- **Stepping:** 5
- **CPU max MHz:** 4600.0000
- **CPU min MHz:** 800.0000
- **BogoMIPS:** 5990.40
- **Virtualization:** VT-x
- **L1d cache:** 288 KiB (6 instances)

- **L1i cache:** 192 KiB (6 instances)
- **L2 cache:** 7.5 MiB (6 instances)
- **L3 cache:** 18 MiB (1 instance)
- **Flags:** fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm epb invpcid_single tpr_shadow vnmi flexpriority ept vpid fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln pts

8.2 Hardware Details for HPC Cluster (Node gics1)

- **Architecture:** x86_64
- **CPU op-mode(s):** 32-bit, 64-bit
- **Byte Order:** Little Endian
- **CPU(s):** 24
- **On-line CPU(s) list:** 0–23
- **Thread(s) per core:** 2
- **Core(s) per socket:** 6
- **Socket(s):** 2
- **NUMA node(s):** 2
- **Vendor ID:** GenuineIntel
- **CPU family:** 6
- **Model:** 63
- **Model name:** Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz
- **Stepping:** 2
- **CPU MHz:** 2907.656
- **BogoMIPS:** 4804.69
- **Virtualization:** VT-x
- **L1d cache:** 32K
- **L1i cache:** 32K

- **L2 cache:** 256K
- **L3 cache:** 15360K
- **NUMA node0 CPU(s):** 0–5,12–17
- **NUMA node1 CPU(s):** 6–11,18–23

9 Problem Description

In this lab, we aim to measure the CPU performance of both the Lab PC and the Cluster by running the Stream Benchmark. This benchmark evaluates memory bandwidth using four operations: Copy, Scale, Add, and Triad. These operations involve reading and writing large arrays, and their performance is limited by memory bandwidth rather than computational power.

To analyze performance, we measured throughput (GB/s) vs. problem size and Bandwidth vs. problem size for each operation. The results were plotted separately for the Lab PC and the Cluster, producing a total of 16 graphs.

We consider a simple particle-based computation with the following setup:

- A two-dimensional square domain of size 1 cm $\times$ 1 cm.
- N_p particles randomly distributed within this domain.
- A conceptual mesh of size $N_x \times N_y$ that defines the spatial extent; however, no mesh-based computation is performed in this assignment.

Each particle is characterized by:

- Position (x, y)
- Velocity v

All particles have the same constant mass m. The kinetic energy of each particle is given by:

$$E = \frac{1}{2}mv^2$$

The purpose of this setup is to generate a memory-intensive workload that enables performance analysis across different working data sizes.

Data Structures

The following arrays of length N_p are used:

- $x[p]$, $y[p]$ — particle positions
- $v[p]$ — particle velocity
- $E[p]$ — particle kinetic energy (output)

The following constant is used:

- m — particle mass (same for all particles)

Initialization

Each particle is initialized using the following procedure:

```
for p = 1 to N_p:
    x[p] = random() * 1.0
    y[p] = random() * 1.0
    v[p] = random()
```

Benchmark Kernels

The following benchmark kernels, similar to the STREAM benchmark, are implemented:

- **Copy:** $x[p] = y[p]$
- **Scale:** $x[p] = a \times y[p]$
- **Add:** $s[p] = x[p] + y[p]$
- **Triad:** $s[p] = x[p] + a \times y[p]$
- **Energy Kernel (optional):** $E[p] = \frac{1}{2}mv[p]^2$

For simplicity, the particle mass is taken as $m = 2$. Here, a denotes a scalar constant.

Benchmarking Methodology

We executed the operations on both the Lab PC and the Cluster, measuring:

- Execution time for varying problem sizes.
- Throughput (GB/s) and Bandwidth for each operation.
- Differences in performance between the Lab PC and the Cluster.

For each case, we generated plots of:

- Throughput (Bytes/s) vs. Problem Size

$$\text{Throughput} = \frac{\text{sizeof(double)} \times N \times \text{Total}}{\text{alg_time}}$$

where N is the number of vectors used in the algorithm.

- Bandwidth (GB/s) vs. Problem Size

$$\text{Bandwidth} = \frac{\text{sizeof(double)} \times N \times \text{Total}}{\text{alg_time} \times 10^9}$$

where N is the number of vectors accessed in the algorithm.

10 Graphical Results

10.1 Vector Copy Operation : $a[i] = b[i]$

10.1.1 Throughput and Bandwidth using Lab PC

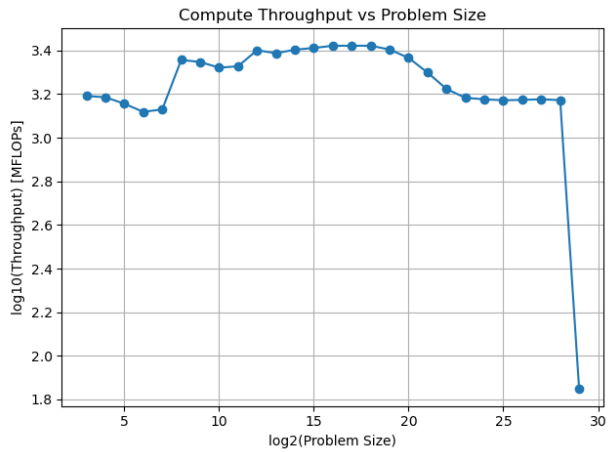

Figure 1: Throughput vs. Problem Size for the vector copy operation using Lab PC.

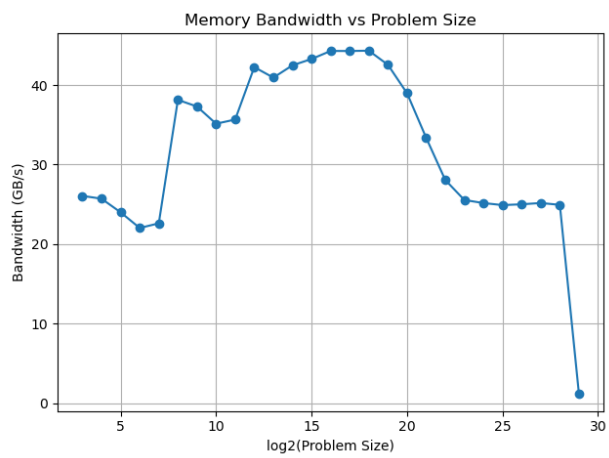

Figure 2: Memory Bandwidth vs. Problem Size for the vector copy operation using Lab PC.

10.1.2 Throughput and FLOPs using HPC Cluster

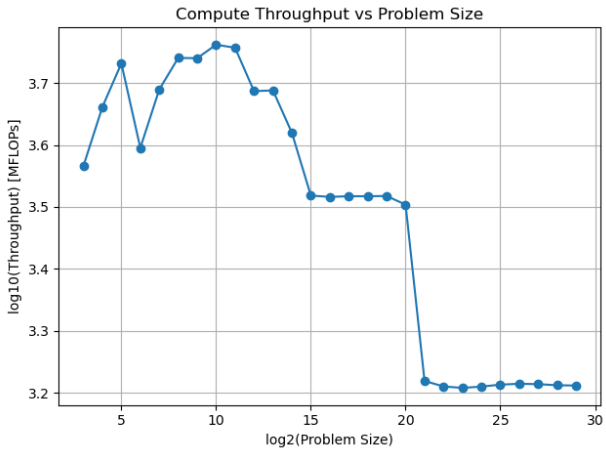

Figure 3: Throughput vs. Problem Size for the vector copy operation using HPC Cluster.

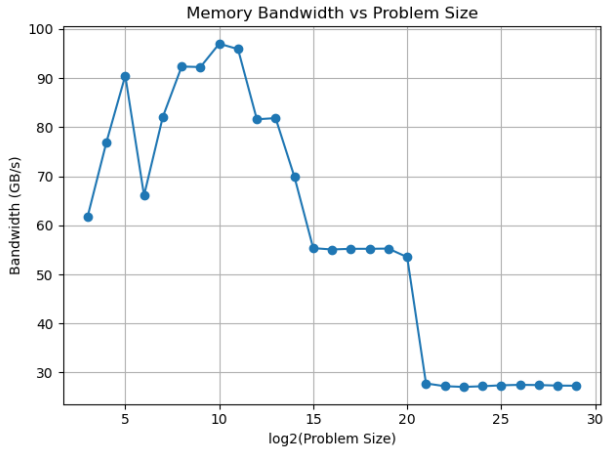

Figure 4: FLOPs vs. Problem Size for the vector copy operation using HPC Cluster.

10.2 Vector Scaling Operation : $a[i] = k * b[i]$

10.2.1 Throughput and Bandwidth using Lab PC

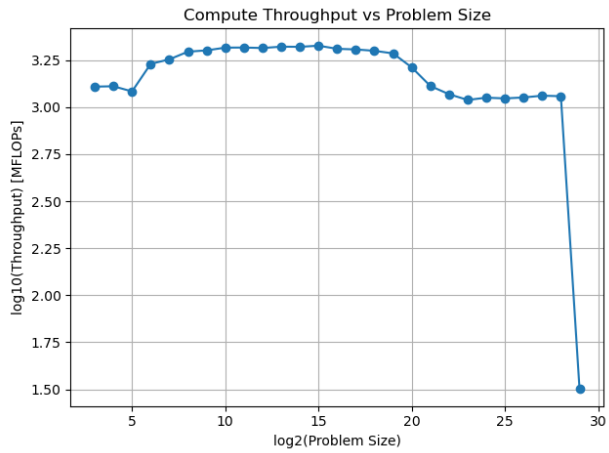

Figure 5: Throughput vs. Problem Size for the vector scaling operation using Lab PC.

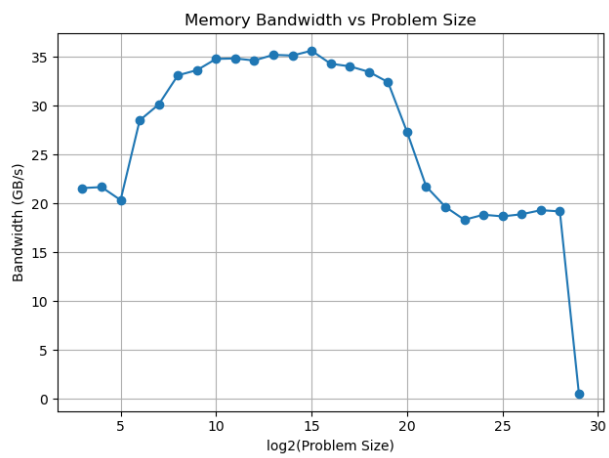

Figure 6: Memory Bandwidth vs. Problem Size for the vector scaling operation using Lab PC.

10.2.2 Throughput and FLOPs using HPC Cluster

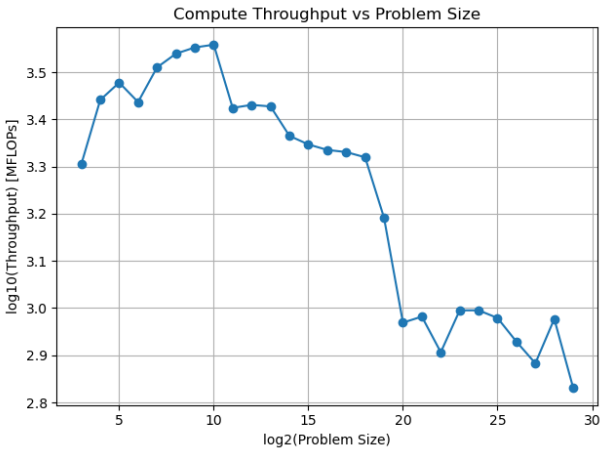

Figure 7: Throughput vs. Problem Size for the vector triad operation using HPC Cluster

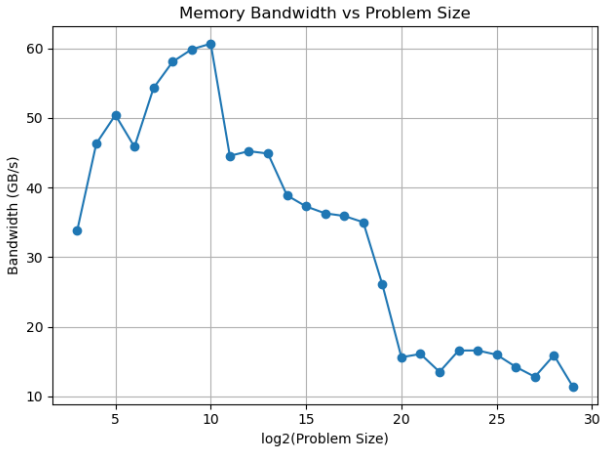

Figure 8: FLOPs vs. Problem Size for the vector triad operation using HPC Cluster.

10.3 Vector Add Operation : $a[i] = b[i] + c[i]$

10.3.1 Throughput and Bandwidth using Lab PC

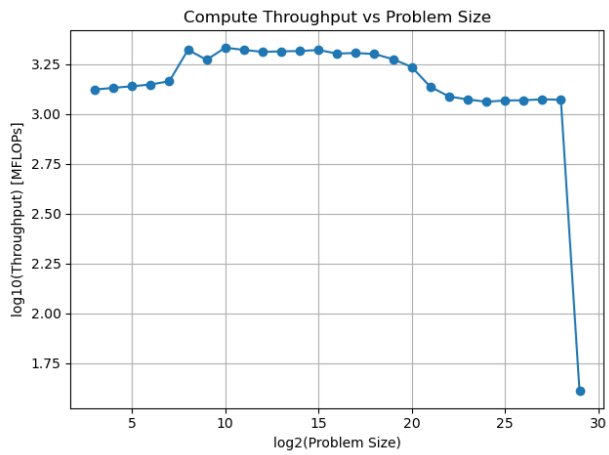

Figure 9: Throughput vs. Problem Size for the vector sum operation using Lab PC.

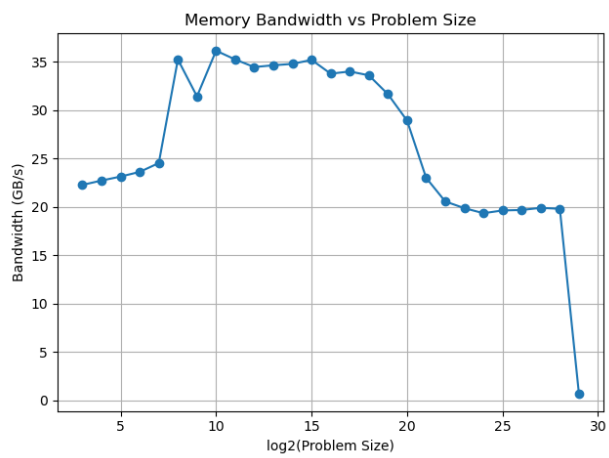

Figure 10: Memory Bandwidth vs. Problem Size for the vector sum operation using Lab PC.

10.3.2 Throughput and FLOPs using HPC Cluster

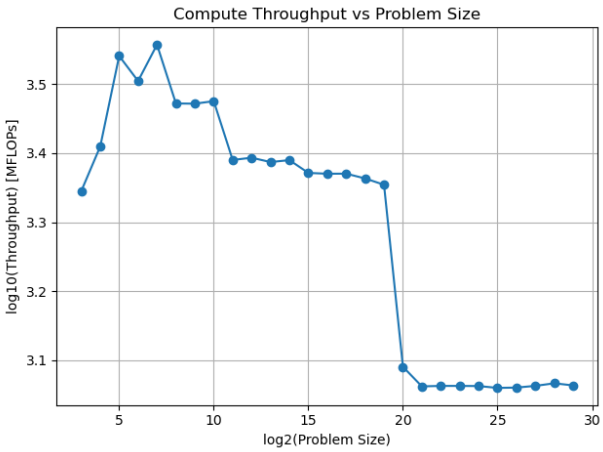

Figure 11: Throughput vs. Problem Size for the vector sum operation using HPC Cluster

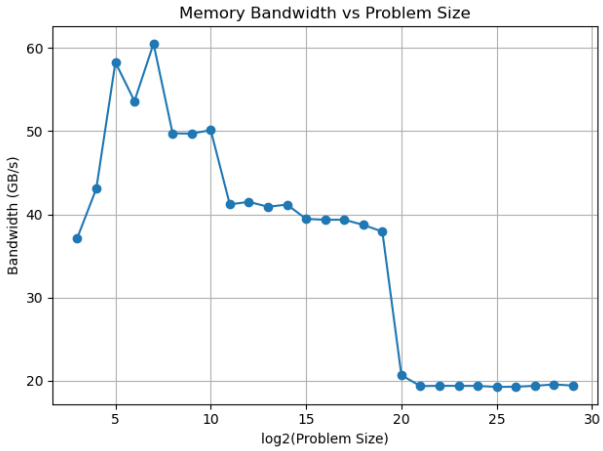

Figure 12: FLOPs vs. Problem Size for the vector sum operation using HPC Cluster.

10.4 Vector Triad Operation : $a[i] = b[i] + c[i] * d[i]$

10.4.1 Throughput and Bandwidth using Lab PC

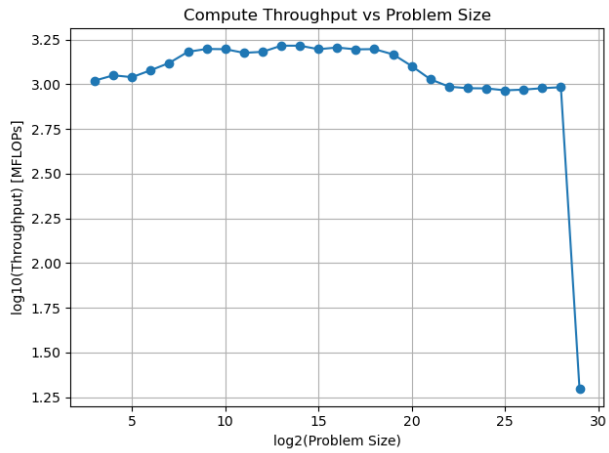

Figure 13: Throughput vs. Problem Size for the vector triad operation using Lab PC.

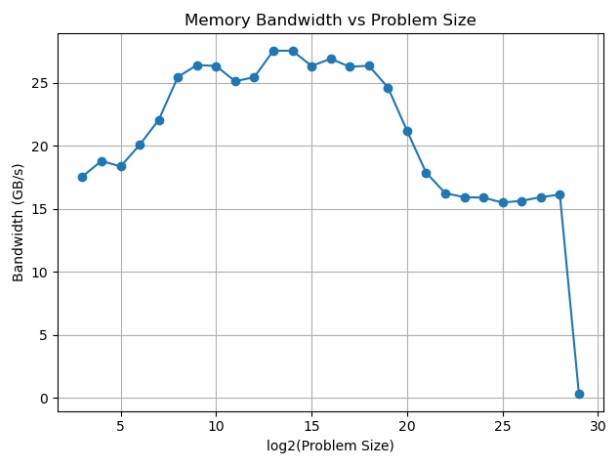

Figure 14: Memory Bandwidth vs. Problem Size for the vector triad operation using Lab PC.

10.4.2 Throughput and FLOPs using HPC Cluster

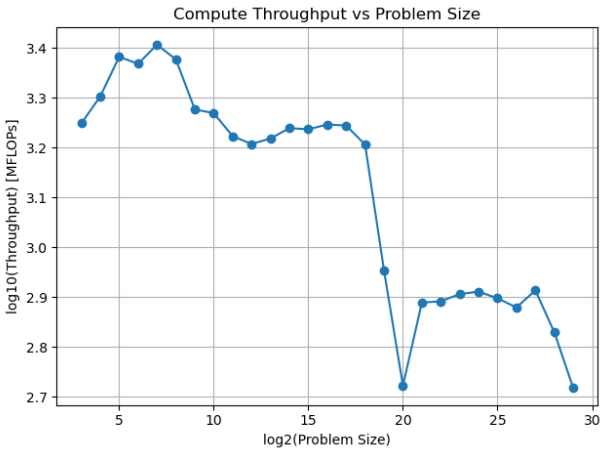

Figure 15: Throughput vs. Problem Size for the vector triad operation using HPC Cluster.

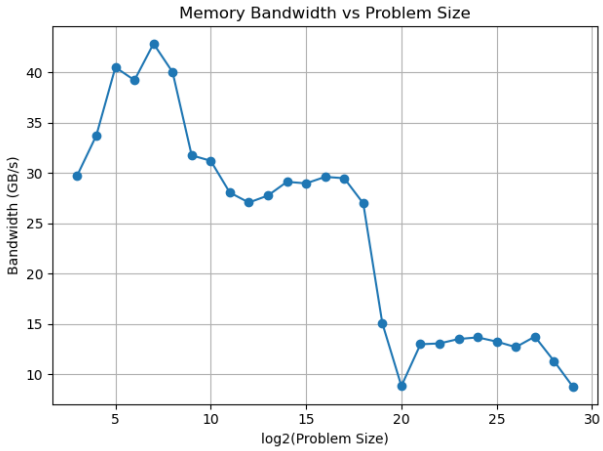

Figure 16: FLOPs vs. Problem Size for the vector triad operation using HPC Cluster.

Analysis Tasks

1. Runtime Measurement and Timing Functions

To measure runtime accurately, high-resolution timing was used in the code. The execution time was measured using the `clock_gettime()` function with the `CLOCK_MONOTONIC` clock. Separate timers were placed around the kernel execution to measure only the algorithm runtime, excluding initialization overhead.

Two commonly used timing functions are explained below:

- `clock()`: This function measures the CPU time consumed by a program. It has low resolution and does not accurately capture delays due to memory access, making it unsuitable for memory-intensive workloads.
- `clock_gettime()`: This function measures wall-clock time with nanosecond resolution. When used with `CLOCK_MONOTONIC`, it provides accurate and reliable timing unaffected by system clock changes. Therefore, it is preferred for performance benchmarking in this lab.

—

2. Bandwidth vs Working Data Size

Bandwidth (GB/s) was plotted against working data size for the four STREAM kernels: Copy, Scale, Add, and Triad. Bandwidth was calculated as the ratio of total data transferred to the kernel execution time.

The plots show that bandwidth remains high for small problem sizes but decreases as the working data size increases. This behavior occurs because data access shifts from cache to main memory, which has lower bandwidth.

—

3. Performance vs Working Data Size

Performance was plotted against working data size for three STREAM kernels. Performance was defined as the inverse of execution time.

The results show that performance is stable for small problem sizes and decreases as data size grows. This trend indicates that memory access time dominates execution as the working set increases.

—

4. Computation Time vs Memory Access Time in the Triad Kernel

In the Triad kernel, each iteration performs one multiplication and one addition, resulting in two floating-point operations. At the same time, it accesses multiple memory locations, including three reads and one write.

The computation time was estimated using the total number of floating-point operations and the theoretical peak compute performance of the CPU. The memory access time was estimated using the total data transferred and the measured memory bandwidth.

The analysis shows that memory access time is significantly larger than computation time, indicating that the Triad kernel is memory-bound.

—

5. Measured vs Theoretical Peak Performance

Theoretical peak performance represents the maximum achievable performance under ideal conditions. However, the measured performance in this lab is lower than the theoretical peak.

This difference occurs due to memory bandwidth limitations, cache misses, and system overheads. Since the STREAM kernels are memory-intensive, the overall performance is constrained by memory speed rather than computational capability.

—

6. Impact of Cache Sizes on Performance

Cache size has a significant impact on performance trends. For small working data sizes, data fits within the CPU caches, resulting in high performance. As the problem size increases and exceeds cache capacity, data must be fetched from main memory, which is slower.

Sudden drops in performance correspond to transitions between cache levels, such as from L1 to L2 or from cache to main memory. This demonstrates the importance of cache hierarchy in determining application performance.

11 Conclusion

We evaluated all four vector operations on both the laboratory system and the HPC cluster. For smaller problem sizes, the performance remained almost constant because the data could fit into the CPU caches, allowing fast access. As the problem size increased, the performance gradually decreased. This happens because the data no longer fits in the cache and must be fetched from main memory, which is much slower. Although the processor is capable of performing computations quickly, the speed of memory access limits the overall performance. As a result, the measured performance is lower than the theoretical peak value. A noticeable drop in performance indicates that the working data size has crossed a cache boundary, causing the system to rely on slower memory accesses.